Chuleta: Git, GitHub y GitHub Actions

Basada en los apuntes de 2º DAW (Despliegue de aplicaciones web): Git básico, ramas y etiquetas, GitHub Pages, fork y pull request, GitHub Actions y Custom Actions.

0. El mapa mental (léelo primero)

- **Git** = programa que guarda el historial de tu código (control de versiones **distribuido**). Todo funciona en tu ordenador.
- **GitHub** = web que aloja repositorios Git (el *remoto*) y añade extras: Pages, forks, pull requests, Actions.
- **GitHub Actions** = automatización (CI/CD) dentro de GitHub. Se define con ficheros **YAML** en `.github/workflows`.
- Flujo diario: **modificas → `git add` → `git commit` → `git push`**.
- Flujo de ramas: **`git checkout -b rama` → trabajas y commit → `git merge`**.

PARTE 1: GIT

1. Sistemas de control de versiones (SCV)

Qué es: sistema que registra los cambios sobre uno o varios archivos a lo largo del tiempo, de modo que se pueden recuperar versiones concretas.

Mínimo que debe ofrecer: (1) almacenamiento de los elementos, (2) posibilidad de cambiarlos, (3) registro histórico de las acciones sobre cada elemento.

Para qué sirve: volver atrás y corregir errores, compartir código, saber quién cambió qué y con qué comentario, y crear **ramas** que luego se mezclan con la principal.

Tipo	Cómo funciona	Pros / Contras	Ejemplos
Local	Copiar archivos a otra carpeta (con fecha/hora) o guardar versiones en una BBDD local	Muy simple, pero **muy propenso a errores**	RCS (macOS)
Centralizado	**Un único servidor** con todos los archivos; los clientes descargan de ahí	Todos saben en qué trabajan los demás; más fácil de administrar. **Punto único de fallo** (si cae, se pierde la historia)	CVS, SVN

Tipo	Cómo funciona	Pros / Contras	Ejemplos
Distribuido	**No hay repositorio central**: cada desarrollador tiene una copia completa con toda la historia; se sincronizan entre sí	Si el servidor muere, cualquier clon lo restaura	**Git**, Mercurial, Bazaar, BitKeeper

CVS (años 80, primer SCV de código abierto, licencia GNU): cliente-servidor; el servidor solo acepta ficheros que estén en la **última versión** (hay que actualizar antes de subir).

SVN / Subversion (año 2000): misma filosofía que CVS pero corrige sus carencias:

- **Commits atómicos** (en CVS un commit interrumpido deja datos inconsistentes).
- Crear ramas es más eficiente: coste **constante** (en CVS es lineal).
- Maneja **archivos binarios** como tal (CVS los trata como texto).
- Envía **incrementos** de los ficheros, no ficheros completos.

2. Fundamentos de Git

- Guarda **instantáneas (snapshots)**, no copias completas ni solo diferencias.
- **Casi todas las operaciones son locales.**
- **Integridad:** no guarda por nombre de archivo, sino por el **hash de su contenido**.
- **3 estados = 3 áreas:**

Estado	Significa	Área
Modified (modificado)	Has cambiado el archivo pero no lo has marcado	Directorio de trabajo
Staged (preparado)	Marcado para ir en el próximo commit	Área de preparación (staging)
Committed (confirmado)	Guardado de forma segura en la BBDD local	Repositorio (carpeta `.git`)

Recorrido: directorio de trabajo --(`git add`)--> área de preparación --(`git commit`)--> repositorio local --(`git push`)--> repositorio remoto.

3. Comandos básicos (repositorio local)

Comando	Qué hace	Opciones importantes
`git init`	Crea un repo en la carpeta actual (crea la carpeta oculta **`.git`**)	`--bare`: repo sin directorio de trabajo, solo como **servidor**
`git add fichero`	Pasa ficheros al **área de preparación** (estado *staged*)	`git add .` (todo) · `-u` solo ficheros **ya rastreados** · `-n` **simula** sin hacer nada
`git commit -m "mensaje"`	Confirma lo que está en preparación (sin `-m` abre el editor)	`--amend` añade cambios al **último commit** (`--no-edit` = sin cambiar el mensaje) · `-am` = `add` + `commit` en un paso
`git status`	Muestra qué está preparado, qué no y qué archivos **no se rastrean**	
`git diff`	Diferencias entre **directorio de trabajo y área de preparación**	`--staged` / `--cached`: entre **preparación y último commit** · `git diff rama1 rama2`: entre dos ramas
`git log`	Historial de commits: **id (hash), autor, fecha y mensaje**	`git log -n` últimos n commits · `--pretty-oneline` (versión resumida, una línea por commit)
`git rm fichero`	Borra el archivo del directorio de trabajo **y** lo añade a preparación. Sirve también para **renombrar** un archivo rastreado	`--cached`: lo saca de preparación (deja de rastrearlo) pero **lo deja en disco** · `-f`: lo elimina de preparación y de trabajo

.gitignore

Fichero con **patrones de lo que Git no debe rastrear**. Qué poner:

- Credenciales y claves de API (se inyectan por **variable de entorno**, nunca al repo).
- Config del editor (`/.vscode`), ficheros de log, ficheros de sistema (`.DS_Store`).
- Carpetas generadas: `/dist`, `/build`.
- Dependencias descargables: `/node_modules`.
- Cobertura de tests: `/coverage`.

GitHub tiene una lista de `.gitignore` por lenguaje: github.com/github/gitignore.

Deshacer cambios: reset vs revert

	`git reset`	`git revert`
Qué hace	**Mueve HEAD y la rama** a la confirmación indicada (reescribe/borra historia)	**No borra** el commit: crea **un commit nuevo** con los cambios inversos
HEAD	Se mueve	**No se mueve** (avanza con el nuevo commit)
Seguridad	Peligroso (`--hard` pierde cambios)	**Más seguro**

Variantes de reset:

- `git reset fichero` → lo saca del área de preparación (deja de rastrearlo).
- `git reset --hard HEAD~1` → elimina el último commit **y pierde los cambios por completo**.
- `git reset --soft HEAD~1` → elimina el último commit pero **los cambios se mantienen en preparación**.

4. Trabajar con remotos

Comando	Qué hace
`git clone url`	Descarga una copia del repo remoto. `git clone url nombreCarpeta` → la carpeta destino se llamará `nombreCarpeta`
`git remote add nombre url`	Añade un remoto nuevo
`git remote -v`	Lista los remotos
`git remote show nombre`	Configuración detallada de ese remoto
`git push remoto rama`	Envía tus commits al remoto. Ej.: `git push origin main`
`git push -u remoto rama`	Push inicial que **asocia** rama local y remota; los siguientes `git push` ya no necesitan indicarlos
`git pull remoto rama`	**Descarga y fusiona** los cambios remotos en local. **Haz commit de tus cambios antes**
`git fetch remoto`	**Solo descarga** los cambios (los deja en una rama oculta), **no los fusiona**

- Al clonar, Git crea automáticamente el remoto llamado **`origin`** apuntando a la URL clonada.
- **`git pull`** = **`git fetch`** + **`git merge`**.

5. Ramas (branches)

- Una rama es una **bifurcación** del código que evoluciona en paralelo a otras.
- Técnicamente, **una rama es solo un puntero a un commit**: crearla no modifica nada más del repositorio.
- **Crear una rama NO te mueve a ella**: hay que hacer `checkout` después.

Comando	Qué hace
`git branch nueva`	Crea la rama (en local)
`git branch -a`	Lista **todas** las ramas (locales y remotas)
`git branch -r`	Lista las ramas **remotas**

Comando	Qué hace
`git branch -vv`	Muestra la **asociación** de cada rama con su origen remoto
`git branch -d nombre`	Elimina la rama **local**
`git push remoto --delete nombre`	Elimina la rama del **remoto**
`git push -u remoto nombre`	Sube la rama nueva al remoto
`git checkout nombre`	Cambia de rama (actualiza el directorio de trabajo y los nuevos commits irán a esa rama)
`git checkout -b nombre`	**Crea la rama y se mueve a ella** en un solo paso
`git merge rama_a_fusionar`	Integra la rama indicada en la rama **actual**

Importante: al clonar solo obtienes el directorio de trabajo de la rama principal. Para traer otra rama remota basta `git checkout nombre_rama` (mismo nombre que la remota).

Precauciones antes de `git merge`:

1. Estar en la **rama receptora** (la que recibe los cambios; si no, `git checkout <rama>`).
2. Receptora y rama a fusionar deben estar **actualizadas** con el remoto (`git fetch` + `git merge`, o `git pull`).

6. Etiquetado (tags)

Marcan puntos importantes del historial, normalmente **versiones de lanzamiento**. Hay dos tipos: **ligeras** y **anotadas**.

Acción	Comando
Listar (orden alfabético)	`git tag`
Buscar con patrón	`git tag -l 'v1.*'`
Crear **ligera**	`git tag nombre_tag`
Crear **anotada**	`git tag -a nombre_tag -m "Mensaje"`
Etiquetado tardío (commit antiguo)	`git tag -a nombre_tag -m "Mensaje" hash_del_commit` (el hash se ve con `git log --pretty-oneline`)
Eliminar en local	`git tag -d nombre_tag`

Acción	Comando
Eliminar en remoto	`git push origin :refs/tags/nombre_tag`
Enviar **una** etiqueta	`git push origin nombre_etiqueta`
Enviar **todas**	`git push origin --tags`
Ver el código de una etiqueta	`git checkout -b nueva_rama nombre_etiqueta`

- **`git push` NO envía las etiquetas por defecto**: hay que enviarlas explícitamente.
- No se puede hacer checkout directo de una etiqueta (no se puede mover): se **crea una rama nueva en esa etiqueta**.

7. GitHub Pages

Permite **publicar una página web** del proyecto. El contenido puede ser un `README.md` o archivos `.html`. Lo habitual (no obligatorio) es usar una rama **`gh-pages`** que solo tenga la documentación/web.

Pasos:

1. Crear el repositorio.
2. Crear el README o la estructura de ficheros HTML y subirlos al remoto (a `gh-pages` o a otra rama).
3. Activarlo en **Settings** del repo: elegir **la rama** donde está el contenido y, opcionalmente, un **tema** (el tema solo funciona si la documentación está en un `README.md`).

8. Colaborar en proyectos: fork y pull request

- **Fork** = **copiar** un repositorio (a tu cuenta) para experimentar libremente **sin afectar al original**. Después lo clonas y trabajas con normalidad.
- Práctica habitual: añadir el repo original como remoto **`upstream`** (el nombre es un estándar, no obligatorio; significa “principal/producción”):

```
git remote add upstream direccion_repositorio_original
git fetch upstream            # traer cambios del original
git checkout master
git merge upstream            # fusionarlos en tu rama
```

- **Pull request (PR)** = proponer al dueño del repo original que **incluya tus cambios**. Pasos:

1. En la web del repo original: **New pull request**.
2. Opción **"compare across forks"** para comparar entre forks. **base fork** = rama (master) del repo original; **head fork** = la rama de tu fork con los cambios.
3. **Create pull request**: rellenas título y explicación (como una issue) y puedes **pedir revisión** a usuarios concretos (normalmente los mantenedores).
4. Esperas a que lo acepten: pasa a estado **"merged"** y los cambios aparecen en el repo original.

9. Buenas prácticas en Git

- Acordar una **metodología** de trabajo: qué ramas hay y cuándo crear nuevas.
- **Mensajes de commit:**
  - Asunto separado del cuerpo por una **línea en blanco**.
  - Asunto **≤ 50 caracteres**, empieza por **mayúscula**, **sin punto final**, en **imperativo**.
  - Cuerpo con líneas de **≤ 72 caracteres**; explica **qué se ha resuelto y cómo**.
- **No abusar de `git add .`**: añadir solo los ficheros con los que has trabajado (evita ficheros "basura" que engordan el repo).
- **No subir commits con código incompleto** o mezclado: compartir código completo.
- **Un commit por funcionalidad o requerimiento** (solo archivos relacionados). Así se puede hacer `revert` sin problemas. Los archivos no relacionados van en otro commit.

10. Resumen rápido de comandos

```
# Crear
git clone url            # clonar repo existente
git init                 # crear repo local
# Cambios locales
git status
git add fichero
git commit -m "Mensaje"
# Historial
git log                  # todos, del más reciente al más antiguo
git log -n               # los últimos n
# Actualizar y publicar
git fetch remoto         # descarga SIN integrar
git pull                 # descarga E integra (merge)
git push origin master   # publica
# Ramas
git branch nombre        # crear
git branch -a            # listar
```

```
git checkout nombre      # moverse
git checkout -b nombre   # crear y moverse
git push origin nombre   # publicar rama
git branch -d nombre     # borrar local
git merge nombre         # fusionar en la rama actual
# Tags
git tag -a nombre -m "Comentario"
git tag -d nombre
git push origin :refs/tags/nombre
git push origin nombre   # una etiqueta
git push origin --tags   # todas
```

PARTE 2: GITHUB ACTIONS

11. ¿Qué es CI/CD?

Término	Definición	Despliega solo a producción
CI – Continuous Integration	Los desarrolladores **integran código en un repo compartido muy a menudo** (varias veces al día). Cada integración se verifica con **tests automáticos** para detectar errores pronto	No
CD – Continuous **Delivery** (entrega continua)	Extensión de CI: el código **siempre está listo** para ir a producción. Cada cambio pasa pruebas y validaciones automáticas, pero **el despliegue no es automático** (se hace cuando el equipo quiere)	**No**
CD – Continuous **Deployment** (despliegue continuo)	Cada cambio aprobado y verificado se **despliega directamente a producción sin intervención humana**. Exige mucha confianza en los tests	**Sí**

Pipeline: Codificar → Build → Testear → Crear release → Desplegar. CI cubre hasta testear; Delivery hasta tener la release lista; Deployment incluye el despliegue automático.

12. GitHub Actions: componentes

Es la herramienta de GitHub para automatizar flujos CI/CD.

Componente	Qué es
Workflow	Procedimiento automatizado añadido a un repo. Tiene **uno o más jobs** y se activa por un **evento** (PR, push a master, etc.)
Job	Conjunto de **steps** que se ejecutan en el **mismo runner**. Por defecto los jobs de un workflow se ejecutan **en paralelo**
Step	**Tarea individual** dentro de un job: puede ser una *action* o un *comando shell*
Action	Comando independiente que se combina para formar steps. Es el **bloque portátil más pequeño** de un workflow
Runner	**Servidor** con la aplicación de ejecución de GitHub Actions instalada; ejecuta los jobs y devuelve el resultado

Jerarquía: Workflow → Jobs → Steps → Actions (o comandos).

Ejemplo de pipeline: *Job CI* = descargar código → preparar node → `npm install` → `npm test`. *Job CD* = descargar código → login en Docker registry → push al Docker registry. Los jobs pueden ser **dependientes o paralelos**.

Tipos de runner

	GitHub-hosted	Self-hosted
Dónde está	En la **nube** de GitHub	**En local** (tu máquina/red)
Mantenimiento	**Cero** (todo configurado)	Hay que **configurarlo y mantenerlo**
Coste	**De pago** por uso (gratis hasta **2000 min/mes**)	**No se paga** por su uso
Flexibilidad	Puede no servir para configuraciones muy específicas	**Completa**; accede a tu red local y recursos propios
SO	**Windows, macOS y Ubuntu (Linux)**	El que tú tengas

13. Crear un workflow (YAML)

- Se escribe en **YAML** (`.yml`) y se guarda en el repo en **`.github/workflows/`**.
- Ejemplo base de los apuntes:

```
name: primer-ejemplo          # (opcional) nombre que sale en la pestaña
Actions
on: [push]                    # evento que dispara el workflow (REQUERIDO)
jobs:                         # agrupa los jobs
  check-bats-version:         # id del job
    runs-on: ubuntu-latest    # runner (REQUERIDO)
    steps:                    # se ejecutan en SECUENCIA
      - uses: actions/checkout@v2     # uses = usar una action
      - uses: actions/setup-node@v1
      - run: npm install -g bats      # run = ejecutar un comando
      - run: bats -v
```

Elementos principales

Clave	Para qué sirve	Detalles a recordar
`name`	Nombre del workflow	**Opcional**. Si falta, GitHub usa la ruta del fichero

Clave	Para qué sirve	Detalles a recordar
`on`	Evento(s) que lo activan	**Requerido**. Puede ser una cadena, una lista, una lista de tipos de evento o un mapa de configuración
`jobs`	Los trabajos	Uno o más; cada uno con un **identificador**; **en paralelo por defecto**
`jobs.<id>.needs`	Qué jobs deben ir **antes**	Evita la ejecución en paralelo. Si un job falla, **sus sucesores no se ejecutan**
`jobs.<id>.runs-on`	Máquina donde corre el job	**Requerido**. GitHub-hosted o self-hosted
`env` / `jobs.<id>.env`	Variables de entorno	Si se define a varios niveles, **gana la más específica**
`jobs.<id>.steps`	Pasos del job	Secuenciales
`steps.uses`	Usar una action	Propia o de terceros (Marketplace)
`steps.run`	Ejecutar un comando de shell	Cada `run` es un shell **independiente**; con `|` se ejecutan varios comandos en el mismo shell. `working-directory` cambia la carpeta
`strategy.matrix`	Ejecutar el **mismo job con varias combinaciones** de variables	Crea varios jobs a partir de una definición

Variantes de `on`

```
on: [push]                        # un evento
on: [push, pull_request]          # varios eventos
on:                               # con filtros
  push:
    branches:
      - main
    tags:
      - v1                        # push al tag v1
  pull_request:
    branches:
      - main
```

`needs` (dependencias entre jobs)

```
jobs:
  job1:
  job2:
    needs: job1                   # job2 espera a job1
  job3:
    needs: [job1, job2]           # job3 espera a ambos
```

Con `if: always()` el job se ejecuta **siempre** tras sus `needs`, **aunque hayan fallado**:

```
  job3:
    if: always()
    needs: [job1, job2]
```

`env` (variables de entorno)

```
env:                    # nivel workflow
  SERVER: production
jobs:
  job1:
    env:                # nivel job
      FIRST_NAME: Mona
    steps:
      - name: Print a greeting
        env:            # nivel step
          MY_VAR: Hi there! My name is
        run: echo $MY_VAR $FIRST_NAME
```

`uses` (actions de terceros y propias)

```
steps:
  - uses: actions/checkout@v2            # action oficial, con versión (@v2)
  - uses: actions/heroku@1.0.0           # rama por defecto o versión
concreta
  - uses: ./.github/actions/my-action    # action PROPIA del repo (ruta
local)
```

`run` (varios comandos y carpeta)

```
- name: Install Dependencies
  run: npm install

- name: Clean install dependencies and build
  run: |
    npm ci
    npm run build

- name: Clean temp directory
  run: rm -rf *
  working-directory: ./temp
```

`strategy.matrix`

```
strategy:
  matrix:
    os: [ubuntu-16.04, ubuntu-18.04]
    node: [6, 8, 10]            # 2 SO x 3 versiones = 6 ejecuciones
runs-on: ${{ matrix.os }}
steps:
  - uses: actions/setup-node@v1
    with:
      node-version: ${{ matrix.node }}
```

Con **`include`** se definen combinaciones concretas, con variables extra por combinación:

```
strategy:
  matrix:
    include:
      - node-version: 10.x
        site: "prod"
        datacenter: "site-a"
      - node-version: 12.x
        site: "dev"
        datacenter: "site-b"
steps:
  - name: Echo site details
    env:
      SITE: ${{ matrix.site }}
      DATACENTER: ${{ matrix.datacenter }}
    run: echo $SITE $DATACENTER
```

- Se accede a los valores con `${{ matrix.variable }}`.
- `with:` pasa **parámetros de entrada** a una action.

14. Custom GitHub Actions (acciones propias)

- Son **tareas individuales** que combinas para crear jobs. Se ejecutan **dentro de un step**, que es parte de un job, que es parte de un workflow.

Tipo	Ventajas	Inconvenientes
Docker-container action	Más **consistente y fiable**; no te preocupas de dependencias/herramientas	**Solo Linux** (runner); **más lentas**
JavaScript action	Código más **simple** y **ejecución más rápida**	Para que corra en **todos los runners** debe ser **JavaScript puro**

Fichero de metadatos (`action.yml` / `action.yaml`)

Define **entradas, salidas y punto de entrada principal**. Elementos:

Elemento	¿Requerido?	Qué es
`name`	**Sí**	Nombre que GitHub muestra en la pestaña Actions
`author`	No	Autor de la action

Elemento	¿Requerido?	Qué es
`description`	**Sí**	Descripción breve
`inputs`	No	Parámetros de entrada. Cada input tiene `description`, `required` y `default`; **todos obligatorios salvo `default`**. GitHub crea una variable de entorno **`INPUT_<NOMBRE>`** (nombre en **MAYÚSCULAS** y **espacios → `_`**)
`outputs`	No	Datos que la action establece. **Todos sus atributos son requeridos**
`runs`	**Sí**	Cómo se ejecuta; **cambia según sea JavaScript o Docker**
`branding`	No	**Color e icono** para el Marketplace de GitHub

`runs` en JavaScript:

```
runs:
  using: 'node12'      # REQUERIDO: aplicación para ejecutar el código (hoy se usa node20)
  pre: 'setup.js'      # opcional: script ANTES de main (condicional con pre-if)
  main: 'index.js'     # REQUERIDO: fichero con el código principal
  post: 'cleanup.js'   # opcional: script DESPUÉS de main (condicional con post-if)
```

`runs` en Docker:

```
runs:
  using: 'docker'               # requerido
  image: 'Dockerfile'           # requerido: Dockerfile local o imagen pública (Docker Hub, otro registro)
  args:                         # entradas para el contenedor
    - 'bzz'
  entrypoint: 'main.sh'         # invalida el ENTRYPOINT del Dockerfile
  post-entrypoint: 'cleanup.sh'
```

Ejemplo completo: action en JavaScript

Pasos: (1) crear `action.yml`, (2) crear proyecto Node (instalar librerías, crear `index.js`, compilar), (3) subir cambios y probar en un workflow.

1. **`action.yml`**, ubicado en `.github/actions/hello-world-javascript-action/action.yml`:

```
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:             # id del input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time:                     # id del output
    description: 'The time we greeted you'
runs:
  using: 'node12'
  main: 'index.js'
```

2. Proyecto Node (en esa misma carpeta):

```
npm init -y
npm install @actions/core
npm install @actions/github
```

`index.js` (lógica principal):

```
const core = require('@actions/core');
const github = require('@actions/github');

try {
  // 'who-to-greet' es el input definido en action.yml
  const nameToGreet = core.getInput('who-to-greet');
  console.log(`Hello ${nameToGreet}!`);
  const time = (new Date()).toTimeString();
  core.setOutput("time", time);
  // Payload JSON del evento que disparó el workflow
  const payload = JSON.stringify(github.context.payload, undefined, 2)
  console.log(`The event payload: ${payload}`);
} catch (error) {
  core.setFailed(error.message);
}
```

Compilar (para **no tener que subir `node_modules`** al repo):

```
npm i -g @vercel/ncc
ncc build index.js --license licenses.txt
# y cambiar en action.yml:  main: 'dist/index.js'
```

3. Probar la action en un workflow:

```
on: [push]
jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    name: A job to say hello
    steps:
      # Hay que hacer checkout para poder usar la action privada del repo
      - name: Checkout
        uses: actions/checkout@v2
      - name: Hello world action step
        uses: ./.github/actions/hello-world-javascript-action   # ruta de la
action
        id: hello                    # id para leer sus outputs después
        with:
          who-to-greet: 'Mona the Octocat'
      - name: Get the output time
        run: echo "The time was ${{ steps.hello.outputs.time }}"
```

Ejemplo completo: action en Docker

Pasos: (1) crear `Dockerfile` (y el script), (2) crear `action.yml`, (3) subir y probar en un workflow.

`Dockerfile`:

```
# Imagen del contenedor que ejecuta tu código
FROM alpine:3.10
# Copia tu archivo de código del repositorio
COPY entrypoint.sh /entrypoint.sh
# Archivo del código a ejecutar cuando empieza el contenedor
ENTRYPOINT ["/entrypoint.sh"]
```

`entrypoint.sh`:

```
#!/bin/sh -l

echo "Hello $1"
time=$(date)
echo "::set-output name=time::$time"
```

`action.yml`:

```
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time:
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Probar en un workflow: igual que en JavaScript, pero con `uses: ./` si la action está en la raíz del repo (`# Uses an action in the root directory`).

PARTE 3: TRUCOS PARA EL EXAMEN

15. Diferencias que suelen preguntar

Pregunta típica	Respuesta corta
`git add` vs `git commit`	`add` → pasa a **preparación**; `commit` → guarda en el **repo local**
`git fetch` vs `git pull`	`fetch` solo **descarga**; `pull` descarga **y fusiona** (`fetch` + `merge`)
`git push` vs `git commit`	`commit` es **local**; `push` **envía al remoto**
`git reset` vs `git revert`	`reset` mueve HEAD (borra historia); `revert` **crea un commit inverso** (más seguro)
`git branch` vs `git checkout`	`branch` **crea** la rama; `checkout` **te mueve** a ella (`checkout -b` hace ambas)
`git rm` vs `git rm --cached`	sin opción: borra del disco y de preparación; `--cached` : **solo deja de rastrear**, el archivo queda en disco
`git diff` vs `git diff --staged`	trabajo ↔ preparación **vs** preparación ↔ último commit

---

Pregunta típica	Respuesta corta
`git reset --soft` vs `--hard`	`--soft`: cambios **se mantienen en preparación**; `--hard`: **se pierden**
Tag ligera vs anotada	Ligera: `git tag nombre`; anotada: `git tag -a nombre -m "msg"`
Fork vs clone	Fork = copia **en GitHub** (tu cuenta); clone = copia **en tu ordenador**
Fork vs branch	Fork = copia de **todo el repo** para no afectar al original; branch = bifurcación **dentro del mismo repo**
Delivery vs Deployment	Delivery: listo para producir, despliegue **manual**; Deployment: despliegue **automático**
Runner GitHub-hosted vs self-hosted	Nube, de pago, sin mantenimiento **vs** local, gratis, lo mantienes tú
Docker action vs JS action	Docker: fiable pero **solo Linux y lenta**; JS: **rápida** y en todos los runners
`uses` vs `run`	`uses` → ejecutar una **action**; `run` → ejecutar un **comando shell**

16. Checklist de datos "de memoria"

- Carpeta de Git: **`.git`**. Carpeta de workflows: **`.github/workflows`**. Carpeta de actions propias (ejemplo): `.github/actions/<nombre>/`.
- Remoto por defecto al clonar: **`origin`**. Remoto del repo original en un fork: **`upstream`**.
- Rama de GitHub Pages recomendada: **`gh-pages`**.
- Jobs: **paralelo por defecto**; steps: **secuencial**; `needs` crea dependencia.
- Obligatorios en un workflow: **`on`**, **`jobs`** (con **`runs-on`** y **`steps`**). Opcional: `name`.
- Obligatorios en `action.yml`: **`name`**, **`description`**, **`runs`**. Opcionales: `author`, `inputs`, `outputs`, `branding`.
- Minutos gratis en runners de GitHub: **2000/mes**. SO disponibles: **Windows, macOS, Ubuntu**.
- Commit: asunto **≤ 50 caracteres**, mayúscula, sin punto, imperativo; cuerpo **≤ 72** por línea.
- Variable de entorno de un input: **`INPUT_NOMBRE`** (mayúsculas, espacios → `_`).

- Las etiquetas **no se suben con `git push`** normal: `git push origin nombre` o `--tags`.

17. Si te piden escribir un workflow desde cero

Plantilla para copiar y adaptar:

```
# Fichero: .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
env:
  ENTORNO: pruebas
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v1
        with:
          node-version: 12.x
      - run: npm install
      - run: npm test
  deploy:
    needs: build                # solo si build termina bien
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: echo "Desplegando..."
```

Orden mental para redactarlo: nombre → evento (`on`) → jobs → `runs-on` → `steps` (primero `checkout`, luego preparar entorno, luego comandos) → `needs` si hay dependencias → `matrix` si piden varias versiones/SO.